

SQL MASTER NOTES

1/20

What is SQL?

① What is SQL?

SQL (Structured Query Language) is a standard language used to communicate with databases.

- Used to create, read, update and delete data.
- Used to manage database structure and security.

② SQL vs Database vs DBMS

SQL = Language to talk to databases.

Database = Organized collection of data.

DBMS = Software used to manage databases.

③ What is MySQL?

MySQL is an open-source relational database management system. It uses SQL to manage data.

④ Popular DBMS

- MySQL
- Oracle
- Microsoft SQL Server
- PostgreSQL
- SQLite

⑤ Why SQL?

- Easy to learn and use
- Works with most DBMS
- Essential for developers, analysts and data professionals
- Used in web apps, mobile apps, enterprise systems and more.



⑥ Basic Database Commands

- CREATE - create objects (DB, Table)
- READ - read or retrieve data
- UPDATE - modify existing data
- DELETE - remove data
- DROP - remove objects

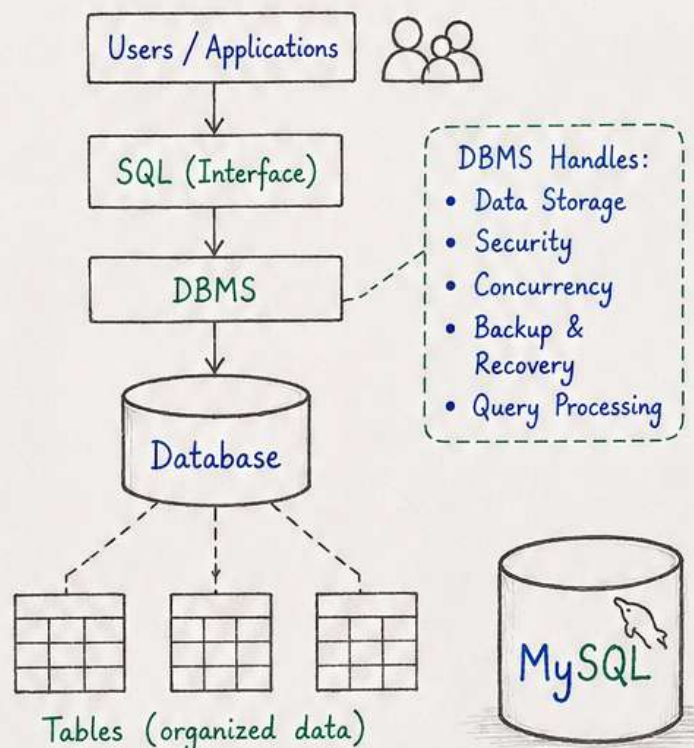
⑦ Comment Styles

- -- single line comment
- /* multi-line comment */

Comments are ignored by SQL.



⑧ DBMS Architecture (Simple View)



★ Quick Note

SQL is not case sensitive, but keywords are usually written in UPPERCASE.



SQL MASTER NOTES

2/20

Getting Ready to Code



① MySQL Server

MySQL Server is the core program that stores, manages and provides access to your data.

- Runs in the background
- Accepts connections
- Executes SQL queries
- Stores data in databases

② MySQL Workbench

MySQL Workbench is a visual tool to connect with MySQL Server.

- Write SQL queries
- Execute queries
- View results
- Manage databases
- Design tables

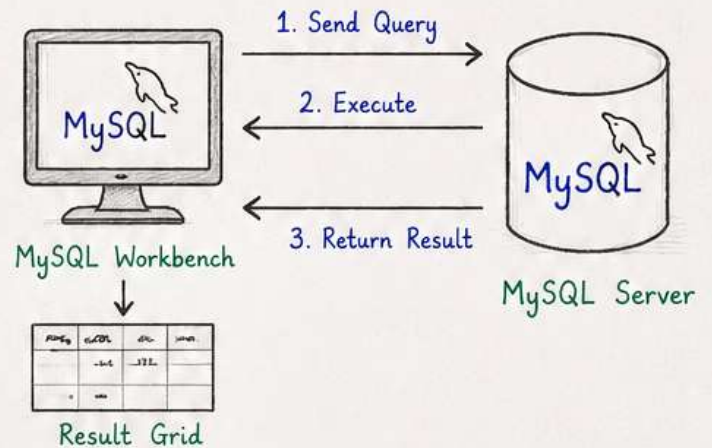
★ Quick Note

MySQL Server does the work, Workbench is just the interface.

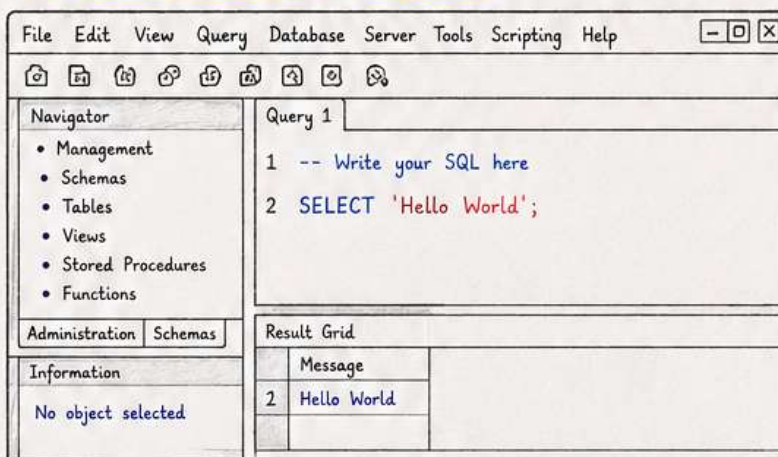


③ How SQL Queries are Executed?

- ① You write a SQL query in Workbench.
- ② Query is sent to MySQL Server.
- ③ Server parses and checks the query.
- ④ Server executes the query.
- ⑤ Results are returned to Workbench.
- ⑥ You see the output in Result Grid.



④ MySQL Workbench Interface (Main Parts)



- Menu Bar → All commands
- Toolbar → Quick actions
- SQL Editor → Write queries here
- Result Grid → Shows output
- Navigator → Databases, tables, objects list

⑤ First SQL Query

```
-- This is a simple query
SELECT 'Hello World' AS Message;
```

Output (Result)

Message
Hello World

⑥ Basic Workflow



SQL MASTER NOTES

3/20

Creating & Using Databases



① What is a Database?

A database is a named collection of organized data or information. It contains related tables, views, procedures and other objects.

② Database Objects

- Tables - store data
- Views - virtual tables (based on queries)
- Queries - saved SQL statements
- Indexes - improve performance
- Stored Procedures - reusable code

③ CREATE DATABASE

Creates a new database.

```
CREATE DATABASE database_name;
```

Example :

- ```
CREATE DATABASE companyDB;
```

### ④ USE DATABASE

Selects a database to work with.

```
USE database_name;
```

Example :

- ```
USE companyDB;
```

⑤ DROP DATABASE

Deletes an existing database.

```
DROP DATABASE database_name;
```

Example :

- ```
DROP DATABASE companyDB;
```

### ⑦ Practical Example (Full Flow)

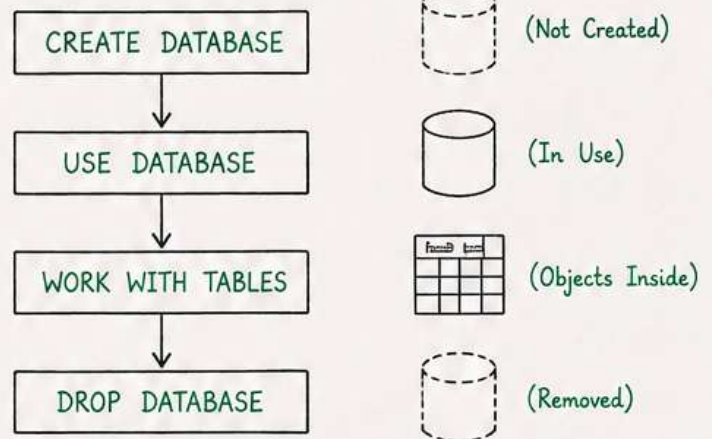
```
-- Step 1 : Create Database
CREATE DATABASE IF NOT EXISTS companyDB;

-- Step 2 : Use Database
USE companyDB;

-- Step 3 : Do your work (create tables, insert data, etc.)

-- Step 4 : Drop Database
DROP DATABASE IF EXISTS companyDB;
```

### Database Life Cycle



### ⑥ IF EXISTS (Safety First!)

Prevents error if the database already exists (for CREATE) or does not exist (for DROP).

```
CREATE DATABASE IF NOT EXISTS companyDB;
```

```
DROP DATABASE IF EXISTS companyDB;
```

#### ★ Quick Note

You must **USE** a database before creating or working with tables in it.

#### ⚠ Safety Note

Be careful with **DROP DATABASE**. It permanently deletes all data and cannot be undone!



# SQL MASTER NOTES

4/20

## Tables, Rows & Columns

### ① What is a Table ?

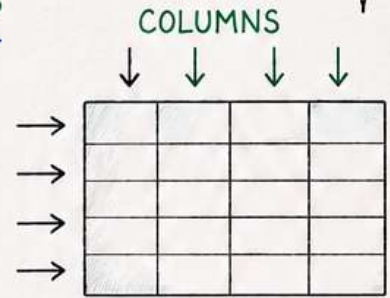
A table is a collection of related data organized in rows and columns. It is like a spreadsheet.

### ② Rows (Records)

Each row in a table represents one record or entry.

### ③ Columns (Fields)

Each column in a table represents one attribute or field. All values in a column are of the same data type.



### ④ Table Structure

| Column Name | Data Type     | Constraints                 | Description     |
|-------------|---------------|-----------------------------|-----------------|
| id          | INT           | PRIMARY KEY, AUTO_INCREMENT | Unique ID       |
| name        | VARCHAR(100)  | NOT NULL                    | Employee Name   |
| department  | VARCHAR(50)   | NOT NULL                    | Department Name |
| salary      | DECIMAL(10,2) | NOT NULL                    | Employee Salary |
| join_date   | DATE          | NOT NULL                    | Date of Joining |

Good table design makes data easy to store, manage and query.



### ⑤ Example: Employee Table (Data)

| id | name        | department | salary   | join_date  |
|----|-------------|------------|----------|------------|
| 1  | Amit Sharma | HR         | 55000.00 | 2024-01-10 |
| 2  | Priya Verma | IT         | 65000.00 | 2024-02-15 |
| 3  | Rohit Singh | Finance    | 60000.00 | 2024-03-20 |
| 4  | Sneha Patel | Marketing  | 58000.00 | 2024-04-05 |

→ One row (record)

→ One column (field)

→ Same data type in a column

#### ★ Quick Note

A table can have any number of rows and columns.



### ⑥ CREATE TABLE Syntax

Used to create a new table in the database.

```
CREATE TABLE table_name (
 column1 datatype constraints,
 column2 datatype constraints,
 column3 datatype constraints,
 ...
 columnN datatype constraints
);
```

#### Common Constraints

- PRIMARY KEY
- NOT NULL
- UNIQUE
- DEFAULT value
- AUTO\_INCREMENT
- FOREIGN KEY

### ⑦ Practical Example

```
CREATE TABLE employees (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(100) NOT NULL,
 department VARCHAR(50) NOT NULL,
 salary DECIMAL(10,2) NOT NULL,
 join_date DATE NOT NULL
);
```



id is the Primary Key (unique for each record)

### ⑧ Good Table Design Tips

- Use meaningful and unique names.
- Choose the correct data type for each column.
- Set PRIMARY KEY for every table.
- Avoid storing unnecessary or duplicate data.
- Normalize tables to reduce redundancy.



# SQL MASTER NOTES

5/20

## Important Data Types



### ① Text / String Data Types

#### ➤ CHAR(size)

Fixed-length string.

Stores exactly the specified number of characters. Pads spaces if needed.

Example: `CHAR(10)`

◦ ABC

#### ➤ VARCHAR(size)

Variable-length string.

Stores up to the specified number of characters.

Uses only the required space.

Example: `VARCHAR(255)`

◦ ABC

### ② Numeric Data Types

#### ➤ INT

Stores whole numbers (positive, negative or zero).

Example: `INT`

◦ 123

#### ➤ FLOAT (Single Precision)

Stores approximate decimal numbers.

Example: `FLOAT`

◦ 12.345

#### ➤ DOUBLE (Double Precision)

Stores larger range and more precise decimals.

Example: `DOUBLE`

◦ 12345.6789

#### ➤ DECIMAL(m,d) (Exact Precision)

Stores exact decimal numbers with fixed precision.

m = total digits, d = digits after decimal.

Best for money, prices, financial calculations.

Example: `DECIMAL(10,2)`

◦ 999.99

### ③ Fixed vs Variable Length Strings

| CHAR                                                                                                                                                                                   | VS | VARCHAR                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>Fixed length</li> <li>Adds spaces if value is smaller</li> <li>Faster for fixed size data</li> <li>Uses more space if value is small</li> </ul> |    | <ul style="list-style-type: none"> <li>Variable length</li> <li>Stores only actual characters</li> <li>Saves storage space</li> <li>Slightly slower than CHAR</li> </ul> |

### ④ DECIMAL(m,d) Explained

DECIMAL(10,2) means:

- Total 10 digits
- 2 digits after decimal
- Remaining 8 digits before decimal

Example: 12345678.90

8 digits before decimal      2 digits after decimal

### ⑤ Best Use Cases (Quick Guide)

| Data Type    | What it Stores        | Best Use Case                        | Example                | Sample Value |
|--------------|-----------------------|--------------------------------------|------------------------|--------------|
| CHAR(n)      | Fixed-length text     | Gender, state code, country code     | Gender CHAR(1)         | 'M', 'F'     |
| VARCHAR(n)   | Variable-length text  | Names, addresses, comments, any text | Full Name VARCHAR(100) | 'John Doe'   |
| INT          | Whole numbers         | Age, counts, IDs, quantities         | Age INT                | 25           |
| FLOAT        | Approximate decimals  | Measurements, percentages            | Height FLOAT           | 175.65       |
| DOUBLE       | More precise decimals | Large range decimals                 | Distance DOUBLE        | 12345.678901 |
| DECIMAL(m,d) | Exact decimals        | Money, prices, financial values      | Salary DECIMAL(10,2)   | 65000.25     |

#### ★ Quick Note

Choose the right data type based on your data. It improves performance, saves storage and avoids data errors.



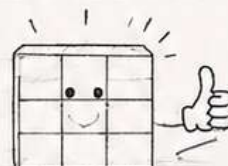
#### Interview Tip

Always prefer the smallest data type that can hold your data. (Don't use BIG types without any reason.)



### ⑥ Real World Example (Employee Table)

```
CREATE TABLE employees (
 id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100) NOT NULL,
 age INT, salary DECIMAL(10,2), join_date DATE);
```



# SQL MASTER NOTES

6/20

## Date / Time Types & Column Constraints



### ① Date / Time Data Types

| Data Type | Format                 | Description                                           | Example                |
|-----------|------------------------|-------------------------------------------------------|------------------------|
| YEAR      | YYYY                   | Stores year value.                                    | 2025                   |
| DATE      | YYYY-MM-DD             | Stores date only.                                     | 2025-05-20             |
| TIME      | HH:MM:SS               | Stores time only.                                     | 14:30:45               |
| DATETIME  | YYYY-MM-DD<br>HH:MM:SS | Stores both date and time.                            | 2025-05-20<br>14:30:45 |
| TIMESTAMP | YYYY-MM-DD<br>HH:MM:SS | Stores date & time with automatic update (see below). | 2025-05-20<br>14:30:45 |

★ Use DATE for birthdays, Time for durations, DATETIME for events, and TIMESTAMP for system tracking.

#### DATETIME vs TIMESTAMP

| DATETIME                                                                                                                                                                                  | TIMESTAMP                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>Range: 1000-01-01 to 9999-12-31</li> <li>No timezone conversion</li> <li>Does not update automatically</li> <li>Use for historical data</li> </ul> | <ul style="list-style-type: none"> <li>Range: 1970-01-01 to 2038-01-19</li> <li>Stored in UTC (timezone aware)</li> <li>Can auto update with CURRENT_TIMESTAMP</li> <li>Use for tracking (created_at, updated_at)</li> </ul> |

### ② Column Constraints

#### ① NOT NULL

Ensures a column must have a value. NULL value is not allowed.

NULL ❌

#### ② UNIQUE

All values in the column must be different. Duplicates are not allowed.

Amit ✓  
Priya ✓  
Amit ❌

#### ③ DEFAULT

Sets a default value when no value is provided.

DEFAULT  
'Active'

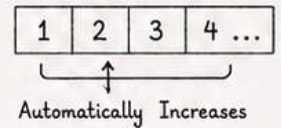
#### ④ PRIMARY KEY

Uniquely identifies each row in a table. It cannot be NULL or DUPLICATE.



#### ⑤ AUTO\_INCREMENT

Automatically increases numeric value for each new row.



### ③ Practical Example (Employee Table with Constraints)

| Column     | Data Type     | Constraints                                              | Description              | Example Value       |
|------------|---------------|----------------------------------------------------------|--------------------------|---------------------|
| id         | INT           | PRIMARY KEY, AUTO_INCREMENT                              | Unique ID for employee   | 1, 2, 3 ...         |
| name       | VARCHAR(100)  | NOT NULL                                                 | Employee name            | Amit Sharma         |
| email      | VARCHAR(100)  | UNIQUE, NOT NULL                                         | Employee email           | amit@gmail.com      |
| department | VARCHAR(50)   | NOT NULL                                                 | Department name          | HR                  |
| salary     | DECIMAL(10,2) | NOT NULL, DEFAULT 0.00                                   | Employee salary          | 55000.00            |
| join_date  | DATE          | NOT NULL                                                 | Date of joining          | 2024-05-20          |
| created_at | TIMESTAMP     | DEFAULT CURRENT_TIMESTAMP                                | Record created time      | 2025-05-20 14:30:45 |
| updated_at | TIMESTAMP     | DEFAULT CURRENT_TIMESTAMP<br>ON UPDATE CURRENT_TIMESTAMP | Record last updated time | 2025-05-20 14:45:10 |

#### Example CREATE TABLE

```
CREATE TABLE employees (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(100) NOT NULL,
 email VARCHAR(100) UNIQUE NOT NULL,
 department VARCHAR(50) NOT NULL,
 salary DECIMAL(10,2) NOT NULL DEFAULT 0.00,
 join_date DATE NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
 ON UPDATE CURRENT_TIMESTAMP
);
```

### ④ DEFAULT CURRENT\_TIMESTAMP

Automatically sets the current date & time.

Example :

```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP;
```



Current  
Date & Time

### ⑤ When to Use ?

- DATE → birthdays, deadlines
- TIME → durations, schedules
- DATETIME → events, logs
- TIMESTAMP → system tracking, auto-updated fields

#### ★ Quick Note

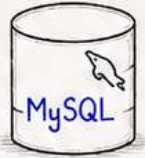
Good use of constraints keeps your data accurate, consistent and secure.



# SQL MASTER NOTES

7/20

## Keys & Relationships



### ① Primary Key

- A primary key uniquely identifies each row in a table.
- It cannot be NULL or DUPLICATE.
- Only one primary key per table.



Example : employees (Parent Table)

| id | name    | department | salary |
|----|---------|------------|--------|
| 1  | Alice   | HR         | 55000  |
| 2  | Bob     | IT         | 65000  |
| 3  | Charlie | Finance    | 60000  |

← id is the Primary Key

### ② Composite Primary Key

- Uses two or more columns together to uniquely identify each row.
- Useful when a single column cannot uniquely identify rows.

Example : order\_items table

| order_id | product_id | quantity | price |
|----------|------------|----------|-------|
| 101      | P1         | 2        | 500   |
| 101      | P2         | 1        | 300   |
| 102      | P1         | 1        | 500   |

← (order\_id, product\_id) together are the Primary Key

### ③ Foreign Key

- A foreign key is a column (or set of columns) that refers to the primary key of another table.
- It helps link two tables and maintain relationships.

Syntax :

```
FOREIGN KEY (column_name)
REFERENCES parent_table(column_name)
[ON DELETE action] [ON UPDATE action];
```

### ⑦ Practical Example (Create Tables with Relationship)

```
CREATE TABLE employees (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(100) NOT NULL,
 department VARCHAR(50)
```

```
);

CREATE TABLE mentorships (
 mentor_id INT,
 mentee_id INT,
 project VARCHAR(100),
 FOREIGN KEY (mentor_id) REFERENCES employees(id)
 ON DELETE CASCADE ON UPDATE CASCADE,
 FOREIGN KEY (mentee_id) REFERENCES employees(id)
 ON DELETE CASCADE ON UPDATE CASCADE
);
```

### ④ Parent Table vs Child Table

- Parent Table : Table that has the PRIMARY KEY.
- Child Table : Table that has FOREIGN KEY referencing the parent.

Parent Table  
(employees)

| id | name    |
|----|---------|
| 1  | Alice   |
| 2  | Bob     |
| 3  | Charlie |

Child Table  
(mentorships)

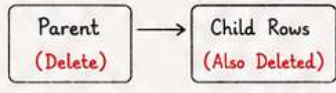
| mentor_id | mentee_id | project          |
|-----------|-----------|------------------|
| 1         | 2         | Website Redesign |
| 1         | 3         | Data Pipeline    |
| 2         | 3         | Mobile App       |

- mentor\_id references employees(id)
- mentee\_id references employees(id)

### ⑤ ON DELETE / ON UPDATE Actions

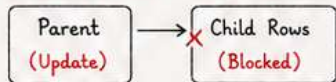
#### ► ON DELETE CASCADE

If a parent row is deleted, all related child rows are automatically deleted.



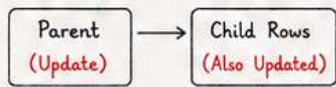
#### ► ON UPDATE RESTRICT

Does not allow update of the parent key if child rows exist.



#### ► ON UPDATE CASCADE

If the parent key is updated, child rows are updated automatically.



### ⑥ Relationship Diagram



1 = One ∞ = Many

= Primary Key  
(FK) = Foreign Key

### ⑧ Quick Interview Notes

#### ★ Quick Interview Notes

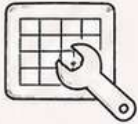
- ✓ Primary Key → Unique identifier of a row
- ✓ Foreign Key → Links two tables
- ✓ Always set proper ON DELETE / ON UPDATE rules
- ✓ Use meaningful keys for better relationships



# SQL MASTER NOTES

8/20

## Changing Tables with ALTER TABLE



### ★ What is ALTER TABLE?

ALTER TABLE is used to change the structure of an existing table. We can add, remove, modify or rename columns, table name, and manage constraints.

#### ① RENAME TABLE

Changes the name of an existing table.

```
RENAME TABLE old_table_name TO new_table_name;
```

Example :

```
• RENAME TABLE employees TO staff;
```

#### ② ADD COLUMN

Adds a new column to the table.

```
ALTER TABLE table_name
ADD COLUMN column_name datatype [constraints] [AFTER column_name];
```

Example :

```
• ALTER TABLE employees ADD COLUMN email VARCHAR(100) AFTER name;
```

#### ③ DROP COLUMN

Removes a column from the table.

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Example :

```
• ALTER TABLE employees DROP COLUMN join_date;
```

#### ④ MODIFY COLUMN

Changes the data type or constraints of a column.

```
ALTER TABLE table_name
MODIFY COLUMN column_name new_datatype [constraints];
```

Example :

```
• ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2) NOT NULL;
```

#### ⑤ CHANGE COLUMN (Also Rename Column)

Changes the name and data type of a column.

```
ALTER TABLE table_name
CHANGE COLUMN old_name new_name new_datatype [constraints];
```

Example :

```
• ALTER TABLE employees CHANGE COLUMN dept department VARCHAR(50);
```

Before Changes

| id | name  | department | salary   |
|----|-------|------------|----------|
| 1  | Amit  | HR         | 55000.00 |
| 2  | Priya | IT         | 65000.00 |
| 3  | Rohan | Finance    | 60000.00 |



After Changes (Example)

| id | name  | email           | age | phone      | salary   |
|----|-------|-----------------|-----|------------|----------|
| 1  | Amit  | amit@gmail.com  | 28  | 9876543210 | 55000.00 |
| 2  | Priya | priya@gmail.com | 26  | 9988776655 | 65000.00 |
| 3  | Rohan | rohan@gmail.com | 30  | 9123456780 | 60000.00 |

Original Table : employees

| id | name  | department | salary   |
|----|-------|------------|----------|
| 1  | Amit  | HR         | 55000.00 |
| 2  | Priya | IT         | 65000.00 |
| 3  | Rohan | Finance    | 60000.00 |

#### ⑥ ADD / DROP CONSTRAINTS

- ADD PRIMARY KEY / UNIQUE / CHECK / FOREIGN KEY
- DROP them if not required

```
ALTER TABLE employees
ADD CONSTRAINT chk_salary CHECK (salary > 0);

ALTER TABLE employees DROP CONSTRAINT chk_salary;
```

#### ⑦ AFTER column\_name

Place a new column after a specific column.

```
• ALTER TABLE employees ADD COLUMN age INT AFTER name;
```

#### ⑧ DESCRIBE Table

Shows the structure (columns) of the table.

```
• DESCRIBE employees;
```



#### ⑨ Multiple ALTER TABLE (Example)

You can perform multiple changes one by one.

```
ALTER TABLE employees
ADD COLUMN phone VARCHAR(15) AFTER email,
MODIFY COLUMN salary DECIMAL(12,2) NOT NULL,
DROP COLUMN department;
```

### ★ Quick Note

Always take a backup before making any changes. 😊

### Good Practices



- Plan changes before applying.
- Test in a safe environment.
- Keep backups of important data.
- Avoid dropping columns containing important data.



# SQL MASTER NOTES

9/20

## Insert, Update & Delete Data



### ① INSERT INTO

Used to insert new records into a table.

#### a) Insert all columns

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

Example :

- Insert one row

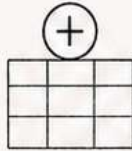
```
INSERT INTO employees
VALUES ('Neha Singh', 'HR', 52000, '2024-05-21');
```

#### b) Insert selected columns

```
INSERT INTO employees (name, department, salary)
VALUES ('Karan Mehta', 'IT', 60000);
```

#### c) Insert multiple rows

```
INSERT INTO employees (name, department, salary)
VALUES
('Ravi Kumar', 'IT', 65000),
('Sneha Patel', 'Marketing', 58000),
('Vikram Raj', 'Finance', 62000);
```



#### ★ NULL Values

Use NULL when no value is available for a column.

Example :

```
INSERT INTO employees
(name, department)
VALUES ('Aman Verma',
NULL);
```



### ② UPDATE

Used to modify existing records.

Syntax :

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

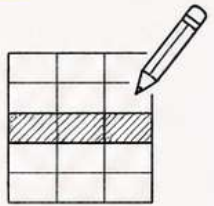
Example :

- Increase salary of employee with id = 2

```
UPDATE employees
SET salary = 68000
WHERE id = 2;
```

- Update multiple columns

```
UPDATE employees
SET department = 'IT', salary = 70000
WHERE id = 3;
```



★ WHERE is very important in UPDATE.  
Without WHERE, all rows will be updated!

### ③ DELETE FROM

Used to remove records from a table.

Syntax :

```
DELETE FROM table_name
WHERE condition;
```

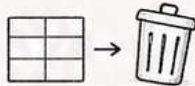
Example :

- Delete employee with id = 3

```
DELETE FROM employees
WHERE id = 3;
```

- Delete multiple rows

```
DELETE FROM employees
WHERE department = 'Marketing';
```



#### ★ Tip

You can also delete all rows using :  
DELETE FROM table\_name;  
(But avoid in production)



### ④ INSERT → UPDATE → DELETE Flow

INSERT

Add new  
data

UPDATE

Modify  
existing data

DELETE

Remove  
old / wrong  
data

#### Employees Table (Current Data)

| id | name        | department | salary   | join_date  |
|----|-------------|------------|----------|------------|
| 1  | Amit Sharma | HR         | 55000.00 | 2024-01-10 |
| 2  | Priya Verma | IT         | 65000.00 | 2024-02-15 |
| 3  | Rohan Mehta | Finance    | 60000.00 | 2024-03-20 |

### ⑤ Example : Complete Operations

- Insert a new employee

```
INSERT INTO employees VALUES ('Arjun Nair', 'IT', 62000, '2024-05-22');
```

- Update his salary

```
UPDATE employees SET salary = 66000 WHERE name = 'Arjun Nair';
```

- Delete an old employee

```
DELETE FROM employees WHERE name = 'Rohan Mehta';
```



#### WARNING

UPDATE or DELETE without WHERE clause can affect ALL rows in the table!  
This can cause huge data loss.  
Always double-check before running queries.



#### Foreign Key Reminder



If a table has FOREIGN KEY constraints, deletion or update may be blocked.  
Follow proper rules or use CASCADE options.



★ Good Practice : Always test queries in a safe environment and backup important data regularly.

# SQL MASTER NOTES

10/20

## SELECT - Reading Data



### ① What is SELECT ?

SELECT is used to read / retrieve data from one or more tables.

```
SELECT column1, column2, ...
FROM table_name;
```

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

Return  
Result

### ② SELECT \*

Returns all columns from the table.

```
SELECT *
FROM employees;
```

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

All Columns

### ③ Selecting Specific Columns

Returns only specified columns.

```
SELECT name, department, salary
FROM employees;
```

| name | department | salary |
|------|------------|--------|
|      |            |        |
|      |            |        |
|      |            |        |

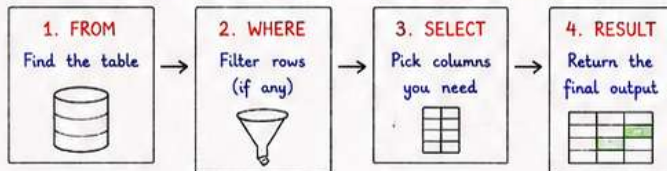
### ④ Column Aliases (AS)

Use aliases to give columns a temporary name.

```
SELECT name AS employee_name,
 salary AS emp_salary
FROM employees;
```

| employee_name | emp_salary |
|---------------|------------|
|               |            |
|               |            |
|               |            |

### ⑨ How SELECT Works ? (Execution Concept)



#### ★ Interview Tip

- ✓ Avoid using SELECT \* in production queries.
- ✓ It fetches unnecessary data.
- ✓ Slows down performance.
- ✓ Uses more bandwidth and memory.
- ✓ Instead, select only the columns you actually need.

### ⑤ DISTINCT

Returns only unique (different) values.

```
SELECT DISTINCT department
FROM employees;
```

| department |
|------------|
| HR         |
| IT         |
| Finance    |
| HR         |
| IT         |

| Result  |
|---------|
| HR      |
| IT      |
| Finance |

### ⑥ LIMIT

Limits the number of rows returned.

```
SELECT *
FROM employees
LIMIT 5;
```

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

Only first  
5 rows  
returned

### ⑦ Selecting Multiple Columns (Example)

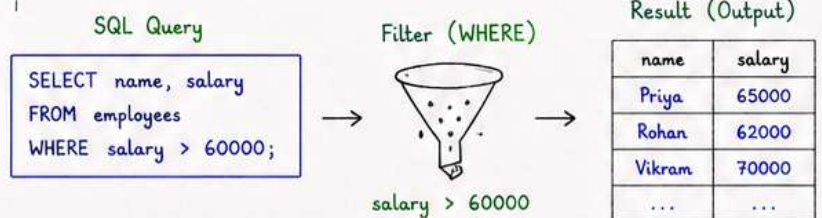
You can select any number of columns you need.

```
SELECT id, name, department, salary, join_date
FROM employees;
```

### ⑧ Practical Examples (Employees Table)

| Query                                                  | Description                           |
|--------------------------------------------------------|---------------------------------------|
| SELECT * FROM employees;                               | Get all data from employees table.    |
| SELECT name, salary FROM employees;                    | Get name and salary of all employees. |
| SELECT DISTINCT department FROM employees;             | Get unique departments.               |
| SELECT name AS emp_name, salary AS sal FROM employees; | Get name and salary with aliases.     |
| SELECT * FROM employees LIMIT 3;                       | Get first 3 rows only.                |

### ⑩ Query → Filter → Result (Visual Flow)



★ Good Practice : Always write clear and optimized SELECT queries. 😊

# SQL MASTER NOTES

11/20

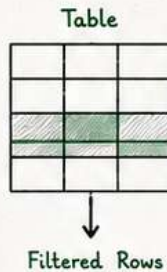
## WHERE & Filtering Data



### ① What is WHERE Clause?

WHERE is used to filter rows based on a condition.

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```



### ② Comparison Operators

| Operator | Meaning               | Example         |
|----------|-----------------------|-----------------|
| =        | Equal to              | salary = 50000  |
| != or <> | Not equal to          | dept != 'HR'    |
| >        | Greater than          | salary > 60000  |
| <        | Less than             | age < 30        |
| >=       | Greater than or equal | salary >= 50000 |
| <=       | Less than or equal    | age <= 28       |

### ③ Logical Operators

- **AND** - Returns true if both conditions are true.
- **OR** - Returns true if at least one condition is true.
- **NOT** - Reverses the result.

### ④ Combining Multiple Conditions

```
SELECT *
FROM employees
WHERE department = 'IT' AND salary > 60000;
```

We can also use OR and NOT

- **OR Example** : WHERE department = 'HR' OR salary > 70000;
- **NOT Example** : WHERE NOT department = 'Finance';

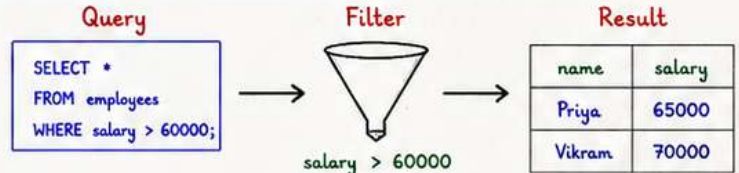
### ⑤ Practical Examples (Employees Table)

| id | name        | department | salary | age |
|----|-------------|------------|--------|-----|
| 1  | Amit Sharma | HR         | 55000  | 28  |
| 2  | Priya Verma | IT         | 65000  | 26  |
| 3  | Rohan Mehta | Finance    | 60000  | 30  |
| 4  | Vikram Raj  | IT         | 70000  | 31  |
| 5  | Neha Singh  | HR         | 52000  | 27  |

### ⑥ WHERE with SELECT (Examples)

| Query                                                                    | Result                                         |
|--------------------------------------------------------------------------|------------------------------------------------|
| SELECT * FROM employees<br>WHERE department = 'IT';                      | All employees from IT department.              |
| SELECT name, salary FROM employees<br>WHERE salary > 60000;              | Employees with salary greater than 60000.      |
| SELECT * FROM employees<br>WHERE salary >= 55000 AND age < 30;           | Employees with salary >= 55000 and age < 30.   |
| SELECT name FROM employees<br>WHERE department = 'HR' OR salary > 60000; | HR employees or those with salary > 60000.     |
| SELECT * FROM employees<br>WHERE NOT department = 'Finance';             | Employees not from Finance department.         |
| SELECT * FROM employees<br>WHERE salary BETWEEN 50000 AND 65000;         | Employees with salary between 50000 and 65000. |

### ⑦ Query → Filter → Result (Execution Flow)



### ⑧ Common Filtering Mistakes

- Using = instead of == (SQL uses single = for comparison).
- Forgetting quotes for text values.  
**Wrong** : WHERE department = IT;  
**Correct** : WHERE department = 'IT';
- Using AND / OR without understanding precedence.
- Missing WHERE clause in UPDATE or DELETE (dangerous!)

⚠ Danger!

UPDATE / DELETE  
without WHERE can  
affect ALL rows in  
the table!



### ⑨ Interview Tip

★ WHERE filters rows **BEFORE** grouping.

Example :  
SELECT department, COUNT(\*)  
FROM employees  
WHERE salary > 60000  
GROUP BY department;

First filter rows where  
salary > 60000, then  
group by department.



### ⑩ Quick Summary

Use WHERE to filter only the rows you need.  
It helps in performance, accuracy, and reduces unnecessary data. 😊



# SQL MASTER NOTES

12/20

## BETWEEN, IN, LIKE & NULL



### ① BETWEEN (For Ranges)

Select values within a range (inclusive).

```
SELECT *
FROM employees
WHERE salary BETWEEN 50000 AND 70000;
```

| id | name   | salary | Result? |
|----|--------|--------|---------|
| 1  | Amit   | 55000  | ✓ Yes   |
| 2  | Priya  | 65000  | ✓ Yes   |
| 3  | Rohan  | 60000  | ✓ Yes   |
| 4  | Vikram | 70000  | ✓ Yes   |
| 5  | Neha   | 52000  | ✓ Yes   |
| 6  | Karan  | 48000  | ✗ No    |

★ BETWEEN  
is inclusive  
of both  
values. 😊

### ② IN / NOT IN

Match any value from a list.

```
SELECT *
FROM employees
WHERE department IN ('HR', 'IT');
```

```
SELECT *
FROM employees
WHERE department NOT IN ('Finance');
```

★ IN =  
matches  
any value  
  
NOT IN =  
excludes  
values 😊

| id | name   | department | Result for IN (HR, IT) |
|----|--------|------------|------------------------|
| 1  | Amit   | HR         | ✓ Yes                  |
| 2  | Priya  | IT         | ✓ Yes                  |
| 3  | Rohan  | Finance    | ✗ No                   |
| 4  | Vikram | IT         | ✓ Yes                  |
| 5  | Neha   | HR         | ✓ Yes                  |
| 6  | Karan  | Marketing  | ✗ No                   |

### ③ LIKE (Pattern Matching)

Used with text columns. Wildcards: % and \_

| Pattern | Meaning         | Example (name) | Matches             |
|---------|-----------------|----------------|---------------------|
| A%      | Starts with A   | A%             | Amit, Arjun, Alice  |
| %a      | Ends with a     | %a             | Priya, Neha, Rohana |
| %an%    | Contains 'an'   | %an%           | Karan, Ankit, Rohan |
| _mit    | _ = single char | _mit           | Amit, Omit (omit)   |
| __ha    | Two chars + ha  | __ha           | Neha, Priha (priha) |

% = any number of characters (including zero)  
\_ = exactly one character 😊

### ④ IS NULL / IS NOT NULL

Used to check for NULL values.

```
SELECT * FROM employees
WHERE manager_id IS NULL;
```

```
SELECT * FROM employees
WHERE manager_id IS NOT NULL;
```

| id | name   | manager_id |
|----|--------|------------|
| 1  | Amit   | NULL       |
| 2  | Priya  | 1          |
| 3  | Rohan  | NULL       |
| 4  | Vikram | 1          |

{ IS NULL → value is missing / not available  
IS NOT NULL → value is available 😊 }

### ⑤ = NULL vs IS NULL

Common mistake! = NULL will never return any rows.

Wrong (No Result)

```
SELECT * FROM employees
WHERE manager_id = NULL;
```

✗ Wrong

Because comparison with NULL  
using = does not work.

Correct (Returns Result)

```
SELECT * FROM employees
WHERE manager_id IS NULL;
```

✓ Correct

Always use IS NULL / IS NOT NULL. 😊

### ⑥ Practical Examples (Employees Table)

Examples :

| Query                                                            | Description                                |
|------------------------------------------------------------------|--------------------------------------------|
| SELECT * FROM employees<br>WHERE salary BETWEEN 50000 AND 70000; | Employees with salary between 50k and 70k. |
| SELECT * FROM employees<br>WHERE department IN ('HR', 'IT');     | Employees from HR or IT department.        |
| SELECT * FROM employees<br>WHERE name LIKE 'A%';                 | Names starting with 'A'.                   |
| SELECT * FROM employees<br>WHERE name LIKE '%an%';               | Names containing 'an' anywhere.            |
| SELECT * FROM employees<br>WHERE manager_id IS NULL;             | Employees who are not managers.            |

### ⑦ Quick Interview Traps

- BETWEEN is inclusive.
- IN is faster & cleaner than using multiple OR.
- LIKE is case-insensitive in MySQL (by default).
- = NULL → never use. Always use IS NULL.
- WHERE filters rows BEFORE GROUP BY / HAVING. 😊

★ Tip

Practice pattern matching with LIKE - it's commonly asked in interviews! 😊

# SQL MASTER NOTES

## ORDER BY & LIMIT

13/20



### ① What is ORDER BY ?

ORDER BY is used to sort the result set in ascending or descending order.

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column_name [ASC | DESC];
```



### ② ASC vs DESC

ASC → Ascending order (default)  
DESC → Descending order

### ③ Sort by Numeric Column (salary)

- Ascending (Low to High)

```
SELECT name, salary
FROM employees
ORDER BY salary ASC;
```

| name   | salary |
|--------|--------|
| Neha   | 52000  |
| Amit   | 55000  |
| Rohan  | 60000  |
| Priya  | 65000  |
| Vikram | 70000  |

- Descending (High to Low)

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

| name   | salary |
|--------|--------|
| Vikram | 70000  |
| Priya  | 65000  |
| Rohan  | 60000  |
| Amit   | 55000  |
| Neha   | 52000  |

### ④ Sort by Text / Date Columns

- By name (A → Z)

```
SELECT name, department
FROM employees
ORDER BY name ASC;
```

| name   | department |
|--------|------------|
| Amit   | HR         |
| Neha   | HR         |
| Priya  | IT         |
| Rohan  | Finance    |
| Vikram | IT         |

- By join\_date (Old → New)

```
SELECT name, join_date
FROM employees
ORDER BY join_date ASC;
```

| name   | join_date  |
|--------|------------|
| Neha   | 2022-04-10 |
| Amit   | 2023-01-05 |
| Rohan  | 2023-07-15 |
| Priya  | 2024-02-11 |
| Vikram | 2024-03-20 |

### ⑤ Multiple Column Sorting

Sort by department (A → Z), then by salary (High → Low)

```
SELECT name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

| name   | department | salary |
|--------|------------|--------|
| Amit   | HR         | 55000  |
| Neha   | HR         | 52000  |
| Priya  | IT         | 65000  |
| Vikram | IT         | 70000  |
| Rohan  | Finance    | 60000  |

### ⑥ What is LIMIT ?

LIMIT is used to restrict the number of rows returned.

Syntax :

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column_name [ASC | DESC]
LIMIT number;
```



Very helpful for Top-N queries!



### ⑦ Top-N Records (Examples)

Top 3 Highest Salaries

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

| name   | salary |
|--------|--------|
| Vikram | 70000  |
| Priya  | 65000  |
| Rohan  | 60000  |

Top 3 Lowest Salaries

```
SELECT name, salary
FROM employees
ORDER BY salary ASC
LIMIT 3;
```

| name  | salary |
|-------|--------|
| Neha  | 52000  |
| Amit  | 55000  |
| Rohan | 60000  |

### ⑧ Latest Records (By Date)

Get latest 3 joined employees.

```
SELECT name, join_date
FROM employees
ORDER BY join_date DESC
LIMIT 3;
```

| name   | join_date  |
|--------|------------|
| Vikram | 2024-03-20 |
| Priya  | 2024-02-11 |
| Rohan  | 2023-07-15 |

DESC → Latest first  
ASC → Oldest first



### ⑨ ORDER BY + LIMIT Combination

Top 2 highest salary in IT department.

```
SELECT name, salary
FROM employees
WHERE department = 'IT'
ORDER BY salary DESC
LIMIT 2;
```

| name   | salary |
|--------|--------|
| Vikram | 70000  |
| Priya  | 65000  |

Remember

ORDER BY is applied first, then LIMIT returns the top rows.

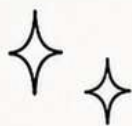


### ★ Interview Tip

Q. Find Top 3 earners in each department.  
Use ORDER BY with LIMIT after filtering the department. 😊

### Good Practice 👍

- Always use ORDER BY with LIMIT for meaningful results.
- Without ORDER BY, LIMIT will return random rows. 😊



# SQL MASTER NOTES

14/20



## Aggregate Functions

### ① What are Aggregate Functions?

Aggregate functions perform a calculation on a set of values and return a **single result**.

```
SELECT AGG_FUNCTION(column_name)
FROM table_name
WHERE condition;
```

★ Always return a single value. 😊

### ② COUNT()

- Returns the number of rows.
- COUNT(\*) counts all rows (including NULLs).
- COUNT(column) counts only non-NULL values.

```
SELECT COUNT(*) FROM employees;
SELECT COUNT(salary) FROM employees;
```

| id | name   | salary | department |
|----|--------|--------|------------|
| 1  | Amit   | 55000  | HR         |
| 2  | Priya  | 65000  | IT         |
| 3  | Rohan  | NULL   | Finance    |
| 4  | Vikram | 70000  | IT         |
| 5  | Neha   | 52000  | HR         |

| Results                                     |
|---------------------------------------------|
| COUNT(*) = 5<br>(All rows)                  |
| COUNT(salary) = 4<br>(Rohan salary is NULL) |

### ③ SUM()

Returns the total (sum) of a numeric column.

```
SELECT SUM(salary) FROM employees;
```

Result

242000

### ④ AVG()

Returns the average value of a numeric column.

```
SELECT AVG(salary) FROM employees;
-- 242000 / 4 = 60500.00
```

Result

60500.00

### ⑤ MIN()

Returns the minimum value.

```
SELECT MIN(salary) FROM employees;
```

Result

52000

### ⑥ MAX()

Returns the maximum value.

```
SELECT MAX(salary) FROM employees;
```

Result

70000

### ⑦ Employee Salary Examples

| id | name   | department | salary |
|----|--------|------------|--------|
| 1  | Amit   | HR         | 55000  |
| 2  | Priya  | IT         | 65000  |
| 3  | Rohan  | Finance    | NULL   |
| 4  | Vikram | IT         | 70000  |
| 5  | Neha   | HR         | 52000  |

Aggregate Results

|               |            |
|---------------|------------|
| COUNT(*)      | = 5        |
| COUNT(salary) | = 4        |
| SUM(salary)   | = 242000   |
| AVG(salary)   | = 60500.00 |
| MIN(salary)   | = 52000    |
| MAX(salary)   | = 70000    |

★ Note: NULL values are ignored in SUM(), AVG(), MIN(), MAX(), and COUNT(column). 😊

### ⑧ Department-wise Calculations (Preview)

We will use GROUP BY to calculate aggregates per department.

```
SELECT department,
COUNT(*) AS total_emp,
AVG(salary) AS avg_salary,
SUM(salary) AS total_salary
FROM employees
GROUP BY department;
```

| department | total_emp | avg_salary | total_salary |
|------------|-----------|------------|--------------|
| HR         | 2         | 53500.00   | 107000       |
| IT         | 2         | 67500.00   | 135000       |
| Finance    | 1         | NULL       | NULL         |

### ⑨ Aggregate vs Normal Functions

Normal Functions (Row by Row)

- Work on each row separately.

```
SELECT name, salary
FROM employees;
```

Returns multiple rows.

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

Aggregate Functions (Set Based)

- Work on a set of rows and return a single value.

```
SELECT SUM(salary)
FROM employees;
```

Returns single value.

|  |
|--|
|  |
|--|

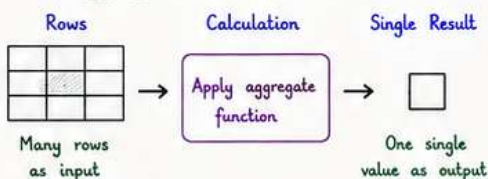
### ⑩ NULL Behavior in Aggregate Functions

- COUNT(\*) → counts all rows (including NULLs).
- COUNT(column) → counts only non-NULL values.
- SUM(), AVG(), MIN(), MAX() → ignore NULL values.
- If all values are NULL, then result is NULL (except COUNT(\*)).

### ⑪ Common Interview Questions

- Write query to find total number of employees.
- Write query to find average salary.
- Write query to find highest salary.
- Write query to find lowest salary.
- Write query to find total salary paid.
- Write query to find department-wise average salary.

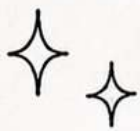
#### Aggregate Function Flow



★ Tip

Aggregate functions are very important in reports, dashboards, analytics and interviews!

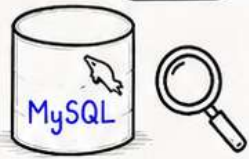




# SQL MASTER NOTES

15/20

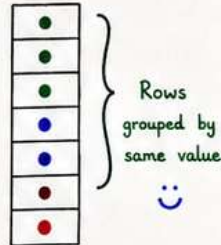
## GROUP BY & HAVING



### ① What is GROUP BY ?

GROUP BY is used to group rows that have the same values.

```
SELECT column_name(s)
FROM table_name
GROUP BY column_name;
```



### ② Grouping with Aggregate Functions

Used with COUNT(), SUM(), AVG(), MIN(), MAX() etc.

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

| department | count |
|------------|-------|
| HR         | 2     |
| IT         | 2     |
| Finance    | 1     |

### ③ COUNT, SUM, AVG with Groups

| department | COUNT(*) | SUM(salary) | AVG(salary) |
|------------|----------|-------------|-------------|
| HR         | 2        | 107000      | 53500.00    |
| IT         | 2        | 135000      | 67500.00    |
| Finance    | 1        | 60000       | 60000.00    |

### ④ What is HAVING ?

HAVING is used to filter groups after aggregation.

```
SELECT department, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```

| department | avg_sal  |
|------------|----------|
| IT         | 67500.00 |
| Finance    | 60000.00 |

### ⑤ WHERE vs HAVING

| WHERE                           | HAVING                              |
|---------------------------------|-------------------------------------|
| Filters rows before grouping.   | Filters groups after grouping.      |
| Cannot use aggregate functions. | Can use aggregate functions.        |
| Used with WHERE clause.         | Used with GROUP BY clause.          |
| Example: WHERE salary > 60000   | Example: HAVING AVG(salary) > 60000 |

### ★ Interview Question

Q: Can we use WHERE with aggregate functions ?

Ans: No! WHERE filters rows before grouping, so aggregate functions are not available. 😊

### ⑥ Filtering Rows vs Filtering Groups

| Filter Rows (WHERE)                                                                                                     | Filter Groups (HAVING)                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Example:</p> <p>Get employees with salary &gt; 60000</p> <pre>SELECT * FROM employees WHERE salary &gt; 60000;</pre> | <p>Example:</p> <p>Get departments with avg salary &gt; 60000</p> <pre>SELECT department, AVG(salary) FROM employees GROUP BY department HAVING AVG(salary) &gt; 60000;</pre> |

### ⑦ Practical Example - Department Wise Salary

Find departments with total salary and average salary, where total salary is greater than 120000.

```
SELECT department,
SUM(salary) AS total_salary,
AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 120000;
```

| department | total_salary | avg_salary |
|------------|--------------|------------|
| IT         | 135000       | 67500.00   |
| Finance    | 60000        | 60000.00   |

### ⑧ Execution Order of SQL Query



Remember: WHERE comes before GROUP BY, HAVING comes after GROUP BY. 😊

### ⑨ Common Mistakes

- Using aggregate functions in WHERE clause. ✗
- Forgetting GROUP BY with aggregate functions. ✗
- Using column in SELECT that is not in GROUP BY or an aggregate function. ✗
- Confusing WHERE and HAVING. ✗

### Wrong Example

```
SELECT department
FROM employees
WHERE AVG(salary) > 60000;
-- Not Allowed 😞
```

### ⑩ More Examples

1) Count employees in each department

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

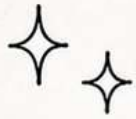
2) Departments with more than 1 employee

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 1;
```

### ⑪ Key Takeaway

Use GROUP BY to group rows.  
Use HAVING to filter the groups based on aggregate results.  
Always follow the correct execution order for accurate results. 😊





# SQL MASTER NOTES

## SQL JOINS

16/20



### ① Why do we need JOINS ?

- JOINS combine data from two or more tables.
- Helps in getting related information.
- Used when data is stored in multiple tables.

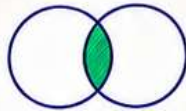


### ② Types of JOINS

| Join Type       | Description              | Shows               |
|-----------------|--------------------------|---------------------|
| INNER JOIN      | Matching rows only       | Common data         |
| LEFT JOIN       | All from left + matches  | Left table + match  |
| RIGHT JOIN      | All from right + matches | Right table + match |
| FULL OUTER JOIN | All rows from both       | All data            |
| CROSS JOIN      | Cartesian product        | All combinations    |

### ③ INNER JOIN

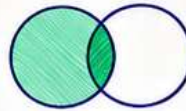
```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN department d
ON e.dept_id = d.id;
```



Returns only matching rows

### ④ LEFT JOIN

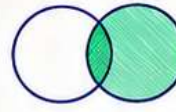
```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN department d
ON e.dept_id = d.id;
```



Returns all rows from left table

### ⑤ RIGHT JOIN

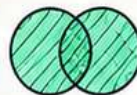
```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN department d
ON e.dept_id = d.id;
```



Returns all rows from right table

### ⑥ CROSS JOIN

```
SELECT e.name, d.dept_name
FROM employees e, department d;
```



Returns Cartesian product (all combinations)

### ⑦ ON Condition

```
FROM table1 JOIN table2 ON table1.id = table2.id
```

- The ON clause specifies the join condition.



### ⑧ JOIN Example (Employees + Department)

| employees |       |         |        |
|-----------|-------|---------|--------|
| id        | name  | dept_id | salary |
| 1         | Amit  | 101     | 55000  |
| 2         | Priya | 102     | 65000  |
| 3         | Rohan | 101     | 60000  |

| department |           |
|------------|-----------|
| id         | dept_name |
| 101        | HR        |
| 102        | IT        |

```
SELECT e.name, d.dept_name, e.salary
FROM employees e
INNER JOIN department d ON e.dept_id = d.id;
```

Output:

| name  | dept_name | salary |
|-------|-----------|--------|
| Amit  | HR        | 55000  |
| Priya | IT        | 65000  |
| Rohan | HR        | 60000  |

### ⑨ JOIN Type Result Comparison

| Join Type       | Shows               | Example Use Case     |
|-----------------|---------------------|----------------------|
| INNER JOIN      | Matching rows only  | Get common data      |
| LEFT JOIN       | All left + matches  | Keep all employees   |
| RIGHT JOIN      | All right + matches | Keep all departments |
| FULL OUTER JOIN | All rows from both  | Show all data        |
| CROSS JOIN      | Cartesian product   | All combinations     |

### ⑩ Practical JOIN Syntax

```
SELECT e.name, d.dept_name
FROM employees e
JOIN department d ON e.dept_id = d.id;
```

★ Tip:

Always specify JOIN condition in ON clause.



### ⑪ Common JOIN Mistakes

- Forgetting the ON condition.
- Using INNER JOIN when LEFT JOIN is required.
- Joining on wrong columns.



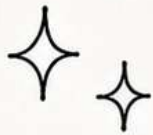
Wrong JOIN can lead to incorrect data!



### ⑫ Most Important JOIN Interview Concepts

- Difference between INNER JOIN and LEFT JOIN.
- When to use RIGHT JOIN.
- How FULL OUTER JOIN works.
- How JOINS improve data retrieval.

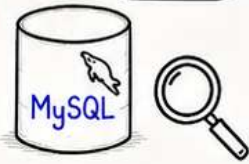




# SQL MASTER NOTES

## SUBQUERIES

18/20



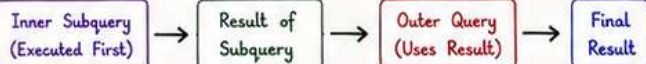
### ① What is a Subquery ?

- A subquery is a query inside another SQL query.
- The **inner query** is executed first,
- and its **result** is used by the **outer query**.

```
SELECT column_names
FROM table_name
WHERE column_name = (
 SELECT column_name
 FROM table_name
);
```

★ Tip:  
Always run  
inner query  
separately first  
to check result.  
😊

### ② Subquery Execution Concept



### ③ Scalar Subquery

- Returns a single value.
- Can be used with =, >, <, >=, <=, != operators.

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Subquery  
returns  
one value.  
😊

Example (Find employees earning more than average salary)

| id | name   | salary |
|----|--------|--------|
| 1  | Amit   | 55000  |
| 2  | Priya  | 65000  |
| 3  | Rohan  | 60000  |
| 4  | Neha   | 52000  |
| 5  | Vikram | 70000  |

AVG(salary)  
= 60400

Result:

| name   | salary |
|--------|--------|
| Priya  | 65000  |
| Vikram | 70000  |

### ④ Subquery with WHERE

- Use subquery in WHERE clause to filter rows.

```
SELECT name, dept_id
FROM employees
WHERE dept_id = (SELECT id FROM departments
 WHERE dept_name = 'HR');
```

★ Tip:  
Subquery gives  
value which  
outer query  
matches.  
😊

### ⑤ IN Subquery

- Use IN to match any value returned by subquery.
- Returns multiple values.

```
SELECT name, dept_id
FROM employees
WHERE dept_id IN (SELECT id FROM departments
 WHERE dept_name IN ('HR', 'IT'));
```

Meaning: get employees who belong to HR or IT department.

### ⑥ EXISTS

- EXISTS checks if subquery returns any rows.
- Returns TRUE if at least one row is returned.

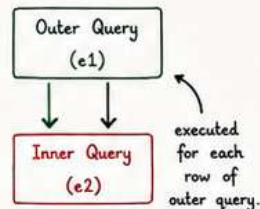
```
SELECT name
FROM employees e
WHERE EXISTS (SELECT 1 FROM departments d
 WHERE d.id = e.dept_id);
```

★ Tip:  
EXISTS is  
faster than  
IN in some  
cases. 😊

### ⑦ Correlated Subquery

- Inner query depends on outer query.
- It is executed once for each row of outer query.

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE e1.salary > (SELECT AVG(e2.salary)
 FROM employees e2
 WHERE e2.dept_id = e1.dept_id);
```



### ⑧ Practical Example: Find employees earning more than average salary in their department

```
SELECT e1.name, e1.dept_id, e1.salary
FROM employees e1
WHERE e1.salary > (SELECT AVG(e2.salary)
 FROM employees e2
 WHERE e2.dept_id = e1.dept_id)
ORDER BY e1.dept_id, e1.salary DESC;
```

employees

| id | name   | dept_id | salary |
|----|--------|---------|--------|
| 1  | Amit   | 101     | 55000  |
| 2  | Priya  | 102     | 65000  |
| 3  | Rohan  | 101     | 60000  |
| 4  | Neha   | 102     | 52000  |
| 5  | Vikram | 101     | 70000  |

Result:

| name   | dept_id | salary |
|--------|---------|--------|
| Rohan  | 101     | 60000  |
| Vikram | 101     | 70000  |
| Priya  | 102     | 65000  |

Dept 101 avg = (55000+60000+70000)/3 = 61666.67

Dept 102 avg = (65000+52000)/2 = 58500

### ⑨ Subquery vs JOIN

| Feature     | Subquery                   | JOIN                        |
|-------------|----------------------------|-----------------------------|
| Readability | Sometimes complex          | Often more readable         |
| Performance | Can be slower              | Generally faster            |
| Use Case    | Good for simple conditions | Better for combining tables |
| Returns     | Single or multiple values  | Combines rows from tables   |

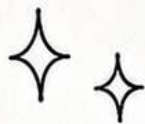
### ⑩ Common Mistakes

- ✗ Using subquery that returns multiple rows with = operator.
- ✗ Not checking subquery result separately.
- ✗ Using IN with NULL values (gives unexpected result).
- ✗ Forgetting correlation in correlated subquery.
- ✗ Overusing subqueries where JOIN is better.

### ★ Interview Questions

- What is a subquery?
- What is correlated subquery?
- Subquery vs JOIN ?
- Write query to find employees earning more than average salary.





# SQL MASTER NOTES

## VIEWS, INDEXES & NORMALIZATION

19/20



### ① What is a VIEW ?

- A VIEW is a virtual table based on the result of a SQL query.
- It **does not** store data physically.

#### CREATE VIEW Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

#### ★ Tip:

Views improve security, simplify complex queries and provide data abstraction. 😊

#### Example

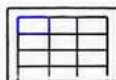
```
CREATE VIEW high_salary_emp AS
SELECT id, name, salary
FROM employees
WHERE salary > 60000;
```

#### Use View

```
SELECT *
FROM high_salary_emp;
```

#### Why Use Views ?

- Simplify complex queries.
- Restrict data access.
- Provide consistent results.
- Useful in reporting.

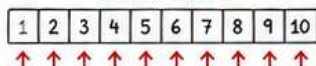


Virtual Table

### ② What is an INDEX ?

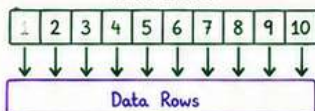
- An index is a data structure that improves the speed of data retrieval operations.
- Works like the index of a book.

#### Without Index



Full table scan (slow)

#### With Index



Uses index first (fast)

#### CREATE INDEX Syntax

```
CREATE INDEX index_name
ON table_name(column_name);
```

Tip: Indexes speed up SELECT queries but slow down INSERT, UPDATE and DELETE. 😊

### ③ Clustered vs Non-Clustered Index

| Clustered Index                                                                                                                                                                        | Non-Clustered Index                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>Sorts the actual data rows.</li><li>Table can have only one.</li><li>Data is stored in sorted order.</li><li>Faster for range queries.</li></ul> | <ul style="list-style-type: none"><li>Sorts the index, not the data.</li><li>Table can have multiple.</li><li>Contains pointer to actual data.</li><li>Good for exact match queries.</li></ul> |
| Index Order → 10 20 30 40<br>↓<br>Data Rows                                                                                                                                            | Index Order → 10 20 30 40<br>Pointers →<br>↓<br>Data Rows                                                                                                                                      |

#### ★ Interview Takeaway

- Use VIEWS for abstraction and security.
- Use INDEXES to improve read performance.
- Normalize your database to reduce redundancy.
- Understand trade-offs between performance and storage.



### ④ What is NORMALIZATION ?

- Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.
- Helps in efficient storage and prevents anomalies.

#### Normalization Forms



### ⑤ Normalization Example

#### Before Normalization (UNF)

| StudentID | StudentName | Courses  | CourseInstructor |
|-----------|-------------|----------|------------------|
| 1         | Amit        | DBMS, OS | John, Mike       |
| 2         | Priya       | DBMS, CN | John, Steve      |
| 3         | Rohan       | OS, CN   | Mike, Steve      |

#### After Normalization (3NF)

| Students  |             | Courses  |            | Enrollments |          |
|-----------|-------------|----------|------------|-------------|----------|
| StudentID | StudentName | CourseID | CourseName | StudentID   | CourseID |
| 1         | Amit        | 101      | DBMS       | 1           | 101      |
| 2         | Priya       | 102      | OS         | 1           | 102      |
| 3         | Rohan       | 103      | CN         | 2           | 101      |
|           |             |          |            | 2           | 103      |
|           |             |          |            | 3           | 102      |
|           |             |          |            | 3           | 103      |

Benefits: No redundancy, better integrity, less storage, easy maintenance. 😊

### ⑥ Practical SQL Example Using Index

```
-- Create index on salary column
CREATE INDEX idx_salary
ON employees(salary);
-- Now this query will be faster
SELECT * FROM employees
WHERE salary > 60000;
```

#### ★ Tip:

Choose columns with high cardinality for indexing (unique or distinct values are higher). 😊

### ⑦ View vs Table

| Feature      | VIEW                       | TABLE             |
|--------------|----------------------------|-------------------|
| Data Storage | Virtual (no storage)       | Physical (stored) |
| Speed        | Slightly slower            | Faster            |
| Security     | Better (restricted access) | Less control      |
| Modification | Limited (depends)          | Full (CRUD)       |

### ⑧ Index vs No Index

| Feature          | With Index | Without Index |
|------------------|------------|---------------|
| Query Speed      | Fast       | Slow          |
| Storage          | More       | Less          |
| Write Operations | Slower     | Faster        |

#### ★ Key Point:

Use indexes smartly. Too many indexes can degrade performance. 😊

